

CNN HYPERPARAMETER TUNING: KERAS TUNER VS. MANUAL TUNING

2D CNN Model — Dogs vs. Cats Image Classification

Frances Frangelico S. Friginal, MIT October 5, 2026

DOES 1C3-DIT / DENG11G1 Prof. Rolando Pula

AI USE DECLARATION

I declare that an AI assistant (Gemini and Gemini Notebook) was used in the preparation of this activity in the following ways:

Guidance — step-by-step hints on which notebook cells to modify so that the source notebook ran on my train, validation, and test splits.

Debugging — identifying an indentation error, a missing comma, and incomplete cell contents encountered during the session.

Verification — checking the values in my comparison table and the figures in my written discussion against the outputs of my own runs.

Document preparation — assembling this document from my completed table, written discussion, and screenshots.

All reported accuracy, loss, and training values were produced by running the notebook myself, and the discussion in Section 6 is my own writing.

1. Project / Experiment Information

Project / Experiment Title	Keras Tuner Hyperparameter Optimization vs. Manual Tuning of a CNN on the Dogs vs. Cats Dataset
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	CNN_Keras_Tuner_FRIGINAL.ipynb (adapted from Loading_Dataset_and_CNN_optimized.ipynb by Rolando A. Pula)
Platform	Google Colab, T4 GPU runtime
Prepared By	Frances Frangelico S. Friginal, MIT
Date	October 2, 2026
Experiment / Trial No.	10 Hyperband trials (search) + 1 final training run

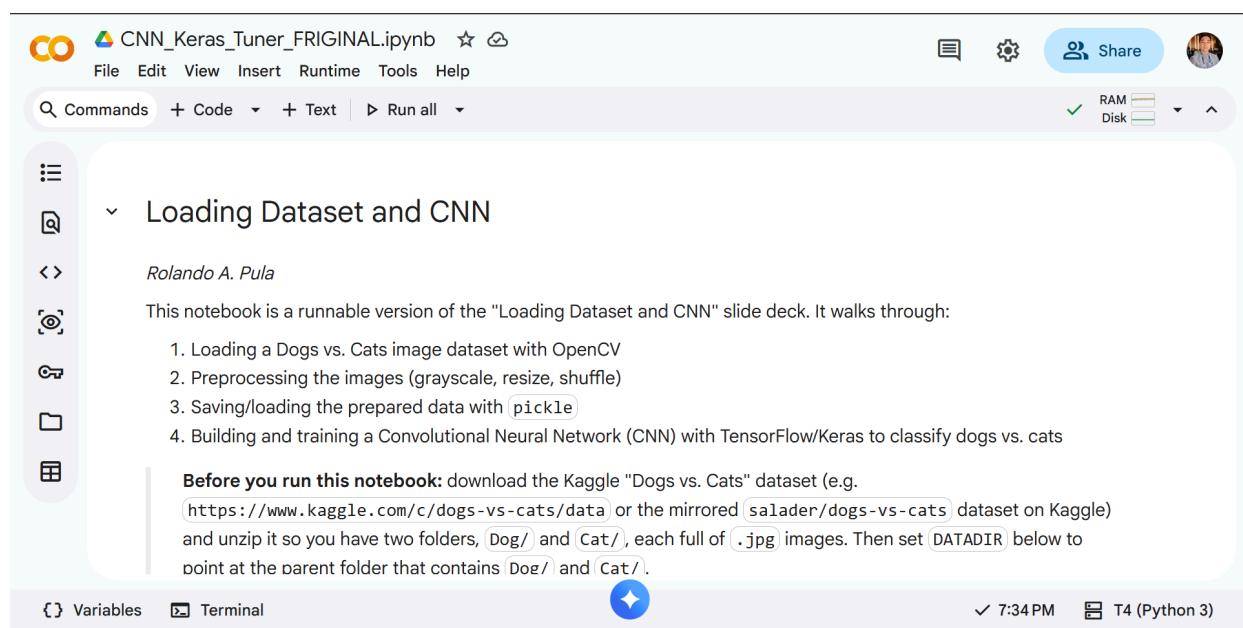


Figure 1. Google Colab notebook and T4 GPU runtime used for this activity.

2. Data and Split

Parameter	Value
Images	24,997 images, resized to 100 × 100, grayscale
Pixel Normalization	X / 255.0, applied once
Train-Test Split	Stratified 80/20, random_state = 42: 19,997 training and 5,000 test images (same split as the manual tuning)
Tuner Validation Split	Training set split again, stratified 80/20, random_state = 42: 15,997 training and 4,000 validation images
Test Set Use	Held out; not used during the search or training; evaluated once after training

3. Keras Tuner Configuration

Setting	Value
Tuner	Hyperband
Objective	val_accuracy
max_epochs / factor	6 / 3 (the notebook default of max_epochs = 15 was reduced to keep the search time manageable)
Trials / Search Time	10 trials, 16 minutes 12 seconds
Search Callback	EarlyStopping (monitor = val_loss, patience = 3)
Final Training	Up to 50 epochs; EarlyStopping (patience = 5, restore_best_weights = True) and ReduceLROnPlateau (factor = 0.5, patience = 2)
Outcome	Training stopped at Epoch 16; weights from Epoch 11 (lowest validation loss, 0.3512) were restored

Search Space — Hyperparameter	Range
Number of convolutional blocks	2 to 4 (step 1)
Filters per convolutional layer	32 to 256 (step 32)
Kernel size per convolutional layer	3 or 5
Number of dense layers	1 to 2
Units per dense layer	32 to 256 (step 32)
Dropout rate	0.0 to 0.5 (step 0.1)
Learning rate	0.0001 to 0.01 (log scale)
Optimizer	Adam or RMSprop
Batch size	32, 64, or 128

4. Comparison: Manual Tuning vs. Keras Tuner

Hyperparameter / Metric	Manual Tuning (Final Model)	Keras Tuner (Best Trial)
1. Architecture	3 Conv2D layers (8 - 64 - 64 filters, 3×3) + 1 Dense (64 units)	4 Conv2D layers (64 - 128 - 32 - 32 filters, 3×3) + 2 Dense (192 - 32 units)
2. Optimizer	Adam	RMSprop
3. Learning Rate	0.001	0.00039186
4. Batch Size	128	32
5. Epochs	40 (fixed)	16 total (stopped by Early Stopping; best weights restored from Epoch 11)
6. Dropout	0.25 (Conv layers) + 0.50 (Dense layer)	0.1
7. Train Accuracy	0.8937	0.9379 (Epoch 11)
8. Validation Accuracy	0.8543	0.8637 (Epoch 11)
9. Test Accuracy	0.8538	0.8628
10. Test Loss	0.3476	0.3518

5. Run Evidence (Screenshots)

```
[26]
✓ 16m
tuner = kt.Hyperband(
    CNNHyperModel(),
    objective="val_accuracy",
    max_epochs=6,
    factor=3,
    directory="kt_dir",
    project_name="dogs_vs_cats_tuning",
    overwrite=True,
)

tuner.search_space_summary()

stop_early = EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)

tuner.search(
    X_tr, y_tr,
    validation_data=(X_val, y_val),
    epochs=6,
    callbacks=[stop_early],
    verbose=1,
)

*** Trial 10 Complete [00h 02m 18s]
val_accuracy: 0.8029999732971191

Best val_accuracy So Far: 0.8240000009536743
Total elapsed time: 00h 16m 12s

[ ]
best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]

print("Best hyperparameters found:")
for param, value in best_hp.values.items():
    print(f" {param}: {value}")
```

Figure 2. Completion of the Hyperband search (Trial 10, total elapsed time 16 min 12 s).

```
Total elapsed time: 00h 16m 12s

[27] ✓ Os  best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]

print("Best hyperparameters found:")
for param, value in best_hp.values.items():
    print(f" {param}: {value}")

*** Best hyperparameters found:
n_conv_layers: 4
filters_0: 64
kernel_size_0: 3
filters_1: 128
kernel_size_1: 3
n_dense_layers: 2
dense_units_0: 192
dropout_rate: 0.1
learning_rate: 0.0003918641276645972
optimizer: rmsprop
tuner/epochs: 6
tuner/initial_epoch: 2
tuner/bracket: 1
tuner/round: 1
filters_2: 32
kernel_size_2: 3
filters_3: 32
kernel_size_3: 3
dense_units_1: 32
batch_size: 32
tuner/trial_id: 0000
```

▼ Training the Tuned Model

We rebuild a fresh model with the winning hyperparameters and train it for longer this time with two callbacks

Figure 3. Best hyperparameters found by Keras Tuner.

```
[28] ✓ 2m  best_model = tuner.hypermodel.build(best_hp)
tuned_batch_size = best_hp.get("batch_size")

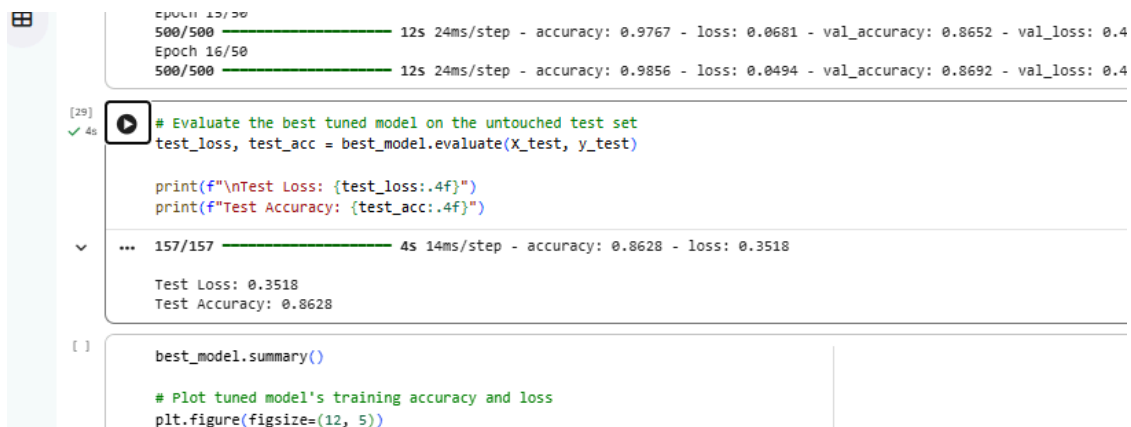
early_stop = EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, min_lr=1e-6)

tuned_history = best_model.fit(
    X_tr, y_tr,
    validation_data=(X_val, y_val),
    batch_size=tuned_batch_size,
    epochs=50,
    callbacks=[early_stop, reduce_lr],
)

*** Epoch 1/50
500/500 — 18s 30ms/step - accuracy: 0.5834 - loss: 0.6660 - val_accuracy: 0.6998 - val_loss: 0.5761 - learning_rate: 3.9186e-04
Epoch 2/50
500/500 — 12s 24ms/step - accuracy: 0.7178 - loss: 0.5502 - val_accuracy: 0.7190 - val_loss: 0.5463 - learning_rate: 3.9186e-04
Epoch 3/50
500/500 — 12s 25ms/step - accuracy: 0.7632 - loss: 0.4927 - val_accuracy: 0.7470 - val_loss: 0.5184 - learning_rate: 3.9186e-04
Epoch 4/50
500/500 — 12s 24ms/step - accuracy: 0.7937 - loss: 0.4438 - val_accuracy: 0.7977 - val_loss: 0.4304 - learning_rate: 3.9186e-04
Epoch 5/50
500/500 — 12s 24ms/step - accuracy: 0.8220 - loss: 0.3970 - val_accuracy: 0.8073 - val_loss: 0.4162 - learning_rate: 3.9186e-04
Epoch 6/50
500/500 — 12s 24ms/step - accuracy: 0.8459 - loss: 0.3574 - val_accuracy: 0.8207 - val_loss: 0.3859 - learning_rate: 3.9186e-04
Epoch 7/50
500/500 — 12s 24ms/step - accuracy: 0.8587 - loss: 0.3273 - val_accuracy: 0.8115 - val_loss: 0.4279 - learning_rate: 3.9186e-04
Epoch 8/50
500/500 — 12s 24ms/step - accuracy: 0.8734 - loss: 0.2948 - val_accuracy: 0.8425 - val_loss: 0.3616 - learning_rate: 3.9186e-04
Epoch 9/50
500/500 — 12s 24ms/step - accuracy: 0.8917 - loss: 0.2564 - val_accuracy: 0.8292 - val_loss: 0.4008 - learning_rate: 3.9186e-04
Epoch 10/50
500/500 — 12s 24ms/step - accuracy: 0.9029 - loss: 0.2322 - val_accuracy: 0.8453 - val_loss: 0.3928 - learning_rate: 3.9186e-04
Epoch 11/50
500/500 — 12s 24ms/step - accuracy: 0.9379 - loss: 0.1617 - val_accuracy: 0.8637 - val_loss: 0.3512 - learning_rate: 1.9593e-04
Epoch 12/50
500/500 — 12s 24ms/step - accuracy: 0.9464 - loss: 0.1407 - val_accuracy: 0.8597 - val_loss: 0.3654 - learning_rate: 1.9593e-04
Epoch 13/50
500/500 — 12s 24ms/step - accuracy: 0.9541 - loss: 0.1200 - val_accuracy: 0.8630 - val_loss: 0.3540 - learning_rate: 1.9593e-04
Epoch 14/50
500/500 — 12s 24ms/step - accuracy: 0.9719 - loss: 0.0804 - val_accuracy: 0.8662 - val_loss: 0.3885 - learning_rate: 9.7966e-05
Epoch 15/50
500/500 — 12s 24ms/step - accuracy: 0.9767 - loss: 0.0681 - val_accuracy: 0.8652 - val_loss: 0.4295 - learning_rate: 9.7966e-05
Epoch 16/50
500/500 — 12s 24ms/step - accuracy: 0.9856 - loss: 0.0494 - val_accuracy: 0.8692 - val_loss: 0.4476 - learning_rate: 4.8983e-05

[ ] best_model.summary()
```

Figure 4. Training log of the tuned model (stopped at Epoch 16, best weights from Epoch 11).



```
Epoch 15/50  
500/500 — 12s 24ms/step - accuracy: 0.9767 - loss: 0.0681 - val_accuracy: 0.8652 - val_loss: 0.4  
Epoch 16/50  
500/500 — 12s 24ms/step - accuracy: 0.9856 - loss: 0.0494 - val_accuracy: 0.8692 - val_loss: 0.4  
[29]  
✓ 4s # Evaluate the best tuned model on the untouched test set  
test_loss, test_acc = best_model.evaluate(X_test, y_test)  
  
print(f"\nTest Loss: {test_loss:.4f}")  
print(f"Test Accuracy: {test_acc:.4f}")  
... 157/157 — 4s 14ms/step - accuracy: 0.8628 - loss: 0.3518  
  
Test Loss: 0.3518  
Test Accuracy: 0.8628  
[ ]  
  
best_model.summary()  
  
# Plot tuned model's training accuracy and loss  
plt.figure(figsize=(12, 5))
```

Figure 5. Evaluation of the tuned model on the held-out test set.

6. Overall Discussion / Remarks

The Keras Tuner model achieved a slightly higher test accuracy of 86.28% compared to the manual model's 85.38% (a 0.90 percentage point improvement), though its test loss was marginally higher at 0.3518 versus 0.3476. However, regarding generalization, the manual model maintained a much tighter train-test gap of 3.99 percentage points (89.37% train vs. 85.38% test), whereas the Keras Tuner model produced nearly double the gap at 7.51 percentage points (93.79% train vs. 86.28% test) likely because the selected architecture utilized a deeper 4-Conv structure alongside lighter regularization (0.1 dropout versus 0.25/0.50), despite actually containing fewer trainable parameters (348,225 vs. 451,409).

In terms of workflow efficiency, manual tuning demanded tedious trial-and-error setup for every iteration, whereas Keras Tuner fully automated the search across candidate architectures within 16 minutes. Ultimately, drawing definitive conclusions from a single run remains statistically limited due to random initialization variance, and neither approach succeeded in reaching Prof. Pula's target threshold of 90% test accuracy.

7. Notes on the Notebook

The Keras Tuner section of the notebook (from the pip install of keras-tuner through the test-set evaluation) was run by me in this session. The earlier cells that show saved outputs without a run number, such as the first baseline training and the sample predictions, carry outputs from the source notebook and are not results of this activity.